IoT Raspberry Pi
Step-by-Step Lab Manual

A separate companion manual for your original MITM IoT Practical PDF (Practical 1–19)

How to use these two PDFs: Keep your original practical PDF open for the program/code. Use this new manual to set up the kit, make connections, install requirements, save the code in the correct place and run it. This manual does not replace or edit your original PDF.

Important limitation: Your uploaded PDF does not identify the exact Raspberry Pi model, sensor board versions, kit wiring diagram or the source repository for the rfid_pi5 module. Therefore, the connections below are standard reference connections for common modules. Before powering the kit, compare the labels printed on your actual module and the lab's pinout sheet. Ask the lab instructor to verify unfamiliar wiring.

Electrical safety: Raspberry Pi GPIO is 3.3V logic and is not 5V tolerant. Never connect a 5V signal directly to a GPIO pin. Do not connect a bulb/mains appliance directly to GPIO or a breadboard. Use a rated, isolated relay module and instructor-supervised mains wiring. For practice, use a low-voltage LED/load.

Credential safety: The Telegram bot token is visible in the supplied PDF. Revoke it with BotFather and create a new token before using Practical 2. Never publish tokens in code screenshots or reports.

Before starting any practical

1. Identify the kit

1. Find the Raspberry Pi board, power supply, microSD card, monitor/HDMI cable, keyboard and mouse (or connect through SSH if your lab uses that).

2. Identify the GPIO header. Use the physical pin numbers printed on a pinout chart; the code's gpiozero LED(17) means BCM GPIO17, not physical pin 17.

3. Identify each module by its printed name and pin labels: VCC, GND, SDA, SCL, CLK, DIO, TX, RX, MOSI, MISO, SCK, SDA/SS, OUT, etc.

4. Keep the Pi powered OFF while making or changing jumper-wire connections. Check every wire before switching power ON.

2. Start Raspberry Pi OS

```
Power on the Raspberry Pi.
Open Terminal from the desktop menu.
Run:
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

3. Create one folder for all practical files

```
mkdir -p ~/iot-practicals
cd ~/iot-practicals
mkdir practical01 practical02 practical03 practical04 practical05 practical06 practical07 practical08 practical09 practical10
```

The folder will be **/home/pi/iot-practicals/** on a typical Raspberry Pi OS account named pi. If your login name is different, the home-folder path will use that username instead. Use `echo $HOME` to see your actual home folder.

4. Python setup (do once)

```
sudo apt install -y python3 python3-pip python3-venv python3-tk python3-dev build-essential
cd ~/iot-practicals
python3 -m venv --system-site-packages myenv
source myenv/bin/activate
python -m pip install --upgrade pip
```

When activated, the prompt normally begins with (myenv). Install the Python libraries for each practical while this environment is active. To activate it in a new terminal: `source ~/iot-practicals/myenv/bin/activate`. To exit: `deactivate`.

5. Enable hardware interfaces

```
sudo raspi-config
# Interface Options:
# Enable I2C for LCD / ADS1115
# Enable SPI for RC522 RFID (if SPI version)
# Enable Serial Port hardware for GPS / fingerprint UART
# Disable serial login shell if using UART for a module
# Enable camera if your OS version offers that setting
sudo reboot
```

6. Basic pin reference (BCM numbering used by gpiozero)

BCM GPIO	Physical header pin	Common use in your PDF
GPIO17	11	LED1 / relay
GPIO18	12	LED2 / MQTT LED / Node-RED LED
GPIO27	13	LED3
GPIO22	15	LED4
GPIO23	16	Node-RED physical push button
GPIO2 / SDA1	3	I2C SDA
GPIO3 / SCL1	5	I2C SCL
GPIO14 / TXD	8	UART TX
GPIO15 / RXD	10	UART RX
GPIO10 / MOSI	19	SPI MOSI

BCM GPIO	Physical header pin	Common use in your PDF
GPIO9 / MISO	21	SPI MISO
GPIO11 / SCLK	23	SPI clock
GPIO8 / CE0	24	SPI CE0 / often RC522 SDA/SS

Power pins: physical pin 1 or 17 = 3.3V; physical pin 2 or 4 = 5V; physical pin 6/9/14/20/25/30/34/39 = GND. Use only the voltage specified by your module.

7. Saving and running a file

```
cd ~/iot-practicals/practical01
nano practical1.py
# paste/type the practical code, then Ctrl+O, Enter, Ctrl+X
python3 practical1.py
```

In Thonny: choose File → Save As → /home/pi/iot-practicals/practical01/practical1.py. Select the Raspberry Pi Python interpreter if prompted, then click Run. Use Terminal for commands beginning with sudo, apt, mkdir, cd, pip, systemctl or python3.

Practical 1 – LED blinking patterns

A. Components required

Raspberry Pi, 4 LEDs, 4 suitable current-limiting resistors (commonly 220–330Ω), breadboard, jumper wires.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
LED anode (+)	GPIO17 physical pin 11 (LED1)	Control signal
LED cathode (–)	Through resistor to GND	Return path
LED2 anode (+)	GPIO18 physical pin 12	Control signal
LED3 anode (+)	GPIO27 physical pin 13	Control signal
LED4 anode (+)	GPIO22 physical pin 15	Control signal
All LED return paths	GND physical pin 6 (common ground)	Complete circuit

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `practical1.py`

D. Install / enable / run

```
sudo apt install -y python3-gpiozero
cd ~/iot-practicals/practical01
source ~/iot-practicals/myenv/bin/activate
python3 practical1.py
```

E. What to check

One LED should turn on at a time in sequence. If none lights, check LED polarity, resistor placement, BCM-to-physical pin mapping and that the Pi is powered.

Practical 2 – Telegram control of Raspberry Pi GPIO

A. Components required

Raspberry Pi with internet, Telegram account/phone, two LEDs, two resistors, breadboard and jumper wires.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
LED1 anode (+)	GPIO17 physical pin 11	Telegram /on_led1 and /off_led1
LED2 anode (+)	GPIO18 physical pin 12	Telegram /on_led2 and /off_led2
LED cathodes (–)	Each through resistor to GND	Return path
Common ground	GND physical pin 6	Circuit ground

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `telegram_led.py`

D. Install / enable / run

```
sudo apt install -y python3-gpiozero
source ~/iot-practicals/myenv/bin/activate
python -m pip install telepot
cd ~/iot-practicals/practical02
```

```
python3 telegram_led.py
```

E. What to check

Start the script and send the supported slash commands to your bot. The terminal should show that the bot is active; the selected LED should switch accordingly.

Important note about the supplied PDF

Your PDF prints a bot token in the source. Do not reuse it. Revoke it and generate a new token. The token line in the PDF also appears to have a quote/syntax issue; enter the new token as a properly quoted string.

Practical 3 – TM1637 four-digit display and clock

A. Components required

Raspberry Pi, TM1637 four-digit display module, four jumper wires.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
VCC	Module-rated supply; check whether your board accepts 3.3V	Power
GND	Pi GND physical pin 6	Ground
CLK	GPIO21 physical pin 40 (first code)	Clock signal
DIO	GPIO20 physical pin 38 (first code)	Data signal

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `tm1637_clock.py`

D. Install / enable / run

```
source ~/iot-practicals/myenv/bin/activate
python -m pip install tm1637
cd ~/iot-practicals/practical03
python3 tm1637_clock.py
```

E. What to check

The display should show the current hour and minute, with the colon blinking. The second brightness example in the PDF uses GPIO23/24; wire CLK/DIO to those pins if running that separate code. Do not mix the two pin configurations.

Practical 4 – GPS module with Raspberry Pi

A. Components required

Raspberry Pi, UART GPS module, jumper wires and a clear view of the sky for satellite reception.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
GPS VCC	Module's specified supply only	Power
GPS GND	Pi GND physical pin 6	Ground
GPS TX	Pi RXD GPIO15 physical pin 10	GPS sends data to Pi
GPS RX	Pi TXD GPIO14 physical pin 8 (often optional)	Pi sends configuration to GPS

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `gps_test.py (filename suggestion only)`

D. Install / enable / run

```
sudo apt update
sudo apt install -y gpsd gpsd-clients python3-serial
# Enable UART in raspi-config, then reboot
ls -l /dev/serial0
# Typical gpsd test (exact setup varies by OS):
sudo systemctl stop gpsd.socket
sudo gpsd /dev/serial0 -F /var/run/gpsd.sock
cgps -s
```

E. What to check

Wait outdoors or near a window for a satellite fix. GPS may output raw NMEA data before a location fix is available.

Important note about the supplied PDF

The uploaded PDF contains only the Practical 4 heading; it has no code, wiring diagram, module model or baud rate. This section is a standard UART GPS setup, not a reconstruction of your missing college program. Confirm your GPS model and program with your instructor.

Practical 5 – ADS1115 analog input / oscilloscope plot

A. Components required

Raspberry Pi, ADS1115 ADC module, analog signal source, jumper wires. Use a safe low-voltage signal only.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
VDD	3.3V physical pin 1 (if module supports 3.3V)	Power
GND	Pi GND physical pin 6	Ground
SDA	GPIO2 physical pin 3	I2C data
SCL	GPIO3 physical pin 5	I2C clock
A0	Low-voltage analog signal output	Analog input channel 0

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `ads_scope.py`

D. Install / enable / run

```
sudo raspi-config   # enable I2C, reboot
sudo apt install -y i2c-tools python3-matplotlib python3-numpy python3-tk
i2cdetect -y 1
source ~/iot-practicals/myenv/bin/activate
# Install the exact ADS1x15 library used by your lab code.
# The PDF's 'from ADS1x15 import ADS1115' is library-specific.
cd ~/iot-practicals/practical05
python3 ads_scope.py
```

E. What to check

A graph window should update as voltage samples arrive; values should also print in Terminal. If address 0x48 does not appear in i2cdetect, check VDD/GND/SDA/SCL and address pins.

Important note about the supplied PDF

The PDF imports ADS1x15. Do not substitute a different ADS1115 package without changing the code to match its API. The code's narrow y-axis limits may make the graph appear blank if the input voltage is outside that range.

Practical 6 – 16x2 LCD over I2C

A. Components required

Raspberry Pi, 16x2 LCD with I2C backpack (usually PCF8574), four jumper wires.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
VCC	LCD backpack-rated supply; verify 3.3V/5V compatibility	Power
GND	Pi GND physical pin 6	Ground
SDA	GPIO2 physical pin 3	I2C data
SCL	GPIO3 physical pin 5	I2C clock

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `lcd_i2c.py`

D. Install / enable / run

```
sudo raspi-config   # enable I2C, reboot
sudo apt install -y i2c-tools
i2cdetect -y 1
source ~/iot-practicals/myenv/bin/activate
python -m pip install RPLCD smbus2
cd ~/iot-practicals/practical06
python3 lcd_i2c.py
```

E. What to check

The first row should show Prachi and the second row should show T.Y.I.T. Use the detected I2C address in the code; the PDF uses 0x27. If the backlight is on but text is missing, adjust the LCD contrast potentiometer.

Practical 7 – RFID RC522 read and write

A. Components required

Raspberry Pi, MFRC522/RC522 RFID reader, compatible RFID card/tag, jumper wires. Use the 3.3V supply for RC522.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
3.3V	Pi 3.3V physical pin 1	Power (do not use 5V)
GND	Pi GND physical pin 6	Ground
SDA / SS	GPIO8 CE0 physical pin 24	SPI chip select
SCK	GPIO11 SCLK physical pin 23	SPI clock
MOSI	GPIO10 MOSI physical pin 19	Pi to reader data
MISO	GPIO9 MISO physical pin 21	Reader to Pi data
RST	GPIO25 physical pin 22 (common example)	Reset

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `WriteRFID.py and ReadRFID.py`

D. Install / enable / run

```
sudo raspi-config   # enable SPI, reboot
sudo apt install -y python3-spidev
# rfid_pi5 is a special third-party module: install from the
# exact repository/instructions provided by your college.
cd ~/iot-practicals/practical07
python3 WriteRFID.py
python3 ReadRFID.py
```

E. What to check

Run WriteRFID.py, enter a short test string, and hold a writable tag near the reader. Then run ReadRFID.py and scan the same tag. A UID and stored text should be printed.

Important note about the supplied PDF

The PDF imports rfid_pi5.SimpleRC522 but does not provide the package URL or install instructions. Exact package installation cannot be determined from the PDF. Confirm the module source and Pi model compatibility with your instructor.

Practical 8 – Fingerprint sensor

A. Components required

Raspberry Pi, UART fingerprint sensor (for example a compatible optical module), jumper wires. Confirm sensor model and voltage first.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
Sensor VCC	Sensor-specified supply; verify before connecting	Power
Sensor GND	Pi GND physical pin 6	Ground
Sensor TX	Pi RXD GPIO15 physical pin 10	Sensor to Pi serial data
Sensor RX	Pi TXD GPIO14 physical pin 8	Pi to sensor serial data

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `fingerprint.py`

D. Install / enable / run

```
sudo raspi-config   # enable UART; disable serial login shell; reboot
ls -l /dev/serial0
source ~/iot-practicals/myenv/bin/activate
python -m pip install pyfingerprint
cd ~/iot-practicals/practical08
python3 fingerprint.py
```

E. What to check

During registration, place and remove the same finger as prompted. During login, place the enrolled finger again. The terminal should report Access Granted or Access Denied.

Important note about the supplied PDF

The PDF hard-codes /dev/ttyAMA0 at 57600 baud. The correct serial device and baud rate depend on the Pi and sensor. If /dev/ttyAMA0 is not the active UART, use the lab's verified device setting; do not blindly change wiring.

Practical 9 – Pi Camera image and video

A. Components required

Raspberry Pi camera module, CSI ribbon cable, Raspberry Pi with camera connector.

B. Kit connections

Power OFF the Pi. Lift the CSI connector latch gently. Insert the ribbon cable fully with contacts oriented as required by the Pi/camera connector, then close the latch. Do not force the cable. Camera connector orientation differs by board; follow the board's printed guide.

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `camera_capture.py`

D. Install / enable / run

```
sudo apt update
sudo apt install -y python3-picamera2
# Check camera detection; command name depends on Raspberry Pi OS release:
rpicam-hello --list-cameras
cd ~/iot-practicals/practical09
python3 camera_capture.py
```

E. What to check

After the code runs, check the current folder for image.jpg and practical_video.mp4. The video duration in the supplied code is five seconds.

Important note about the supplied PDF

Some Raspberry Pi OS releases use rpicam-* commands while older ones use libcamera-*. Use the command available on your installed OS. Picamera2 requires a supported camera stack.

Practical 10 – Home automation with relay

A. Components required

Raspberry Pi, compatible relay module, low-voltage lamp/LED load for lab demonstration, jumper wires and suitable external supply for the load.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
Relay VCC	Module-rated supply; check board specification	Power
Relay GND	Pi GND physical pin 6 (if common-ground input module)	Ground/reference
Relay IN	GPIO17 physical pin 11	Control signal
Load terminals	COM and NO/NC as required by the circuit	Switched load; instructor must wire

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `home_automation.py`

D. Install / enable / run

```
sudo apt install -y python3-gpiozero
cd ~/iot-practicals/practical10
python3 home_automation.py
```

E. What to check

Enter 1 to turn the relay-controlled load on, 2 to turn it off, and 3 to exit. The relay should return to OFF when the program exits.

Important note about the supplied PDF

Do not wire household AC mains yourself. The PDF uses active_high=False, so the relay board's input logic must match that assumption. Relay module input requirements vary.

Practical 11 – RFID with 16x2 LCD message

A. Components required

Raspberry Pi, RC522 RFID reader and tag, 16x2 LCD with I2C backpack, jumper wires.

B. Kit connections

Use the Practical 7 RC522 SPI wiring and Practical 6 LCD I2C wiring together. Ensure SPI and I2C are enabled. Avoid using conflicting pins; these buses use different GPIO pins.

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `WriteRFID.py and ReadRFID.py`

D. Install / enable / run

```
sudo raspi-config   # enable SPI and I2C, reboot
sudo apt install -y i2c-tools python3-spidev
source ~/iot-practicals/myenv/bin/activate
python -m pip install RPLCD smbus2
# Install rfid_pi5 from your college's supplied source.
cd ~/iot-practicals/practical11
python3 WriteRFID.py
python3 ReadRFID.py
```

E. What to check

Write the expected text Guru to a tag using the write program, then run the read program and scan it. The LCD should show Login Successfull if the read text equals Guru; otherwise Login Failed.

Important note about the supplied PDF

The heading in the supplied PDF says fingerprint sensor + LCD, but the actual code uses RFID + LCD. Follow the code's actual hardware unless your instructor supplies a corrected fingerprint program.

Practical 12 – Fingerprint-controlled door lock using relay

A. Components required

Raspberry Pi, UART fingerprint sensor, relay module, low-voltage demonstration lock/load, jumper wires.

B. Kit connections

Connect fingerprint sensor as in Practical 8. Connect relay IN to GPIO17 physical pin 11, relay GND to common Pi GND and module power according to its label. Use an instructor-approved low-voltage load for testing.

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `fingerprint_door_lock.py`

D. Install / enable / run

```
sudo apt install -y python3-gpiozero
source ~/iot-practicals/myenv/bin/activate
```

```
python -m pip install pyfingerprint
cd ~/iot-practicals/practical12
python3 fingerprint_door_lock.py
```

E. What to check

Register the finger, then scan it. A match should activate the relay; an unmatched finger should print Access Denied and leave the relay inactive.

Important note about the supplied PDF

Do not attach a real door lock or mains load without instructor supervision. The PDF code uses active_low relay logic and the UART path /dev/ttyAMA0.

Practical 13 – MQTT publisher-subscriber LED

A. Components required

Raspberry Pi, LED, resistor, breadboard and jumper wires. Mosquitto broker software.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
LED anode (+)	GPIO18 physical pin 12 through appropriate control arrangement	Subscriber output
LED cathode (–)	Through resistor to GND	Return path
Broker	Same Pi, localhost port 1883	Message routing

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `Subscriber.py and Publisher.py`

D. Install / enable / run

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable --now mosquitto
source ~/iot-practicals/myenv/bin/activate
python -m pip install paho-mqtt gpiozero
# Terminal 1:
cd ~/iot-practicals/practical13
python3 Subscriber.py
# Terminal 2 (open a second terminal):
source ~/iot-practicals/myenv/bin/activate
cd ~/iot-practicals/practical13
python3 Publisher.py
```

E. What to check

In the publisher terminal type ON or OFF. The subscriber receives the message on home/pi/led and switches the LED. Keep Subscriber.py running while testing Publisher.py.

Practical 14 – Flask web dashboard for relay

A. Components required

Raspberry Pi, relay module and a low-voltage demonstration load. Raspberry Pi and phone/laptop on the same Wi-Fi network.

B. Kit connections

Connect relay input to GPIO17 physical pin 11, common ground and module-rated supply as in Practical 10. Create the folder and template exactly as shown below.

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `app.py and templates/index.html`

D. Install / enable / run

```
sudo apt install -y python3-gpiozero
source ~/iot-practicals/myenv/bin/activate
python -m pip install Flask
cd ~/iot-practicals/practical14
mkdir -p templates
# Save app.py in practical14/
# Save index.html inside practical14/templates/
python3 app.py
# Find Pi address with: hostname -I
# On phone/laptop open: http://PI-IP-ADDRESS:5000
```

E. What to check

The browser should show Raspberry Pi Light Control and the current status. Press Turn Light ON/OFF and confirm the relay changes. Keep the Flask terminal open while using the page.

Important note about the supplied PDF

The supplied PDF's app.run line appears incomplete. If copied exactly and it causes a syntax error, ask your teacher for the corrected closing parenthesis. The HTML file must be inside the templates folder.

Practical 15 – Two-factor authentication: RFID + fingerprint + relay

A. Components required

Raspberry Pi, RC522 RFID reader and tag, UART fingerprint sensor, relay module and low-voltage demonstration load.

B. Kit connections

Connect RC522 as in Practical 7, fingerprint sensor as in Practical 8, and relay IN to GPIO17 physical pin 11. Enable SPI and UART. Keep all grounds common where the module design requires it.

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `two_factor_auth.py`

D. Install / enable / run

```
sudo apt install -y python3-gpiozero python3-spidev
source ~/iot-practicals/myenv/bin/activate
python -m pip install pyfingerprint
# Install rfid_pi5 from college-provided source.
cd ~/iot-practicals/practical15
python3 two_factor_auth.py
```

E. What to check

Register a fingerprint, scan the enrolled finger and scan the approved RFID tag. The relay should turn on only when both flags are True.

Important note about the supplied PDF

The shown code imports RPLCD but does not initialise or write to an LCD. The PDF also contains an RFID string comparison; use the exact approved text expected by your teacher. This practical uses AND logic, not OR.

Practical 16 – Camera motion detection

A. Components required

Raspberry Pi camera module and CSI ribbon cable. The supplied code also requires NumPy.

B. Kit connections

Connect the camera with Pi powered OFF, using the CSI connector and correct ribbon orientation for your Pi board. This practical's shown program does not use an LCD.

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `motion_detector.py`

D. Install / enable / run

```
sudo apt install -y python3-picamera2 python3-numpy
cd ~/iot-practicals/practical16
python3 motion_detector.py
```

E. What to check

The terminal should display Camera Motion Detector is Active. When movement changes enough pixels, a file named img_.jpg should be saved in the current folder.

Important note about the supplied PDF

The practical title says camera and 16x2 display, but the supplied code only performs camera frame comparison and saves images. No LCD code is present.

Practical 17 – Node-RED dashboard, push button and LED

A. Components required

Raspberry Pi, physical push button, LED and resistor, breadboard, jumper wires, Node-RED, Dashboard 2.0 and Raspberry Pi GPIO nodes.

B. Kit connections

Component pin	Raspberry Pi connection	Purpose
Push-button terminal 1	GPIO23 physical pin 16	GPIO input
Push-button terminal 2	GND physical pin 6	Button return; configure pull-up
LED anode (+)	GPIO18 physical pin 12 through resistor	GPIO output
LED cathode (–)	GND physical pin 6	Return path

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `Node-RED flow (not a Python file)`

D. Install / enable / run

```
# Install Node-RED using its official Raspberry Pi instructions:
# https://nodered.org/docs/getting-started/raspberrypi
node-red-start
# In browser open:
http://PI-IP-ADDRESS:1880
# Install Dashboard 2.0 and Raspberry Pi GPIO nodes via Palette Manager.
# Build/connect nodes, click Deploy, then open:
http://PI-IP-ADDRESS:1880/dashboard
```

E. What to check

Dashboard switch should control LED. Pressing the physical button should force LED ON for five seconds, then restore the dashboard's last stored state. Keep Node-RED running.

Important note about the supplied PDF

The PDF gives node settings and JavaScript function logic, not a complete importable flow. Build the nodes in the editor. The GPIO node packages and hardware support can differ by Raspberry Pi OS/Node-RED version.

Practical 18 – CPU, memory and temperature monitoring

A. Components required

Raspberry Pi only; internet connection for package installation. No external sensor is required by the shown code.

B. Kit connections

None. This practical reads system information from the Raspberry Pi itself.

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `system_monitor.py`

D. Install / enable / run

```
sudo apt update
sudo apt install -y python3-psutil python3-gpiozero
cd ~/iot-practicals/practical18
python3 system_monitor.py
```

E. What to check

Terminal should repeatedly display CPU percentage, RAM percentage and CPU temperature in degrees Celsius. Stop the infinite loop with Ctrl+C.

Practical 19 – Access-control prototype

A. Components required

At minimum, the code shown requires a compatible fingerprint sensor and relay module. The written aim additionally mentions RFID, LCD, LED and event logging.

B. Kit connections

Fingerprint sensor: use Practical 8 UART wiring. Relay: use Practical 10 relay wiring. The PDF code does not show wiring or software use for RFID, LCD, LED status or logging, despite the practical statement mentioning them.

C. Prepare and save the program

Open Thonny (if installed) or a text editor on the Raspberry Pi. Create/open the folder **/home/pi/iot-practicals**. Save this practical's program there using the filename shown below. Keep the original code from your college PDF; do not paste a different program unless your teacher asks.

Suggested filename: `access_control.py`

D. Install / enable / run

```
sudo apt install -y python3-gpiozero
source ~/iot-practicals/myenv/bin/activate
python -m pip install pyfingerprint
cd ~/iot-practicals/practical19
python3 access_control.py
```

E. What to check

The code shown in the PDF registers a fingerprint, searches for a match and pulses the relay for five seconds. It does not show the requested RFID OR logic, LCD feedback, separate LED indication or event log.

Important note about the supplied PDF

The requirement says RFID OR fingerprint, while Practical 15 uses both (AND). These are different access rules. Ask your instructor which version is expected for assessment. Do not claim features that are absent from the code.

Troubleshooting – read this before asking why it does not run

Problem / error	What to check
ModuleNotFoundError	Activate the same virtual environment used for installation; install the exact library in that environment.
No module named rfid_pi5	The PDF does not give its source. Ask instructor for the exact package/repository; do not guess a package name.
I2C device not found	Enable I2C, check SDA/SCL, VCC/GND, module address and run i2cdetect -y 1.
RFID reader not detected	Enable SPI; check 3.3V supply, SPI wiring, CE0/SS, MOSI, MISO, SCLK and GND.
Fingerprint sensor not responding	Enable UART, disable serial login shell, check TX-to-RX crossover, baud rate and device path.
Camera not detected	Power off and reseat ribbon cable with correct orientation; check camera compatibility and OS camera tools.
LED does not light	Check LED polarity, resistor, common ground, BCM pin vs physical pin and whether the script is still running.
Relay works opposite to expected	Relay may be active-LOW or active-HIGH; check module label and code's active_high setting.
Flask page not opening	Keep app running; check hostname -I, same Wi-Fi, port 5000 and firewall/network isolation.
MQTT publisher has no effect	Start Mosquitto and Subscriber first; confirm both use localhost and the exact same topic.
Graph window does not appear	Check python3-tk, display session, Matplotlib backend and whether you are running locally rather than headless SSH.

Useful commands to remember

```
# Where am I?
pwd

# List files in current folder
ls

# Move into practical folder
cd ~/iot-practicals/practical01

# Create a file with Nano
nano practical1.py

# Run Python program
python3 practical1.py

# Stop an infinite loop
Ctrl + C

# Find Raspberry Pi IP address
hostname -I

# Check I2C devices
i2cdetect -y 1

# Check serial device
ls -l /dev/serial0

# Check SPI device nodes
ls /dev/spidev*

# Leave virtual environment
deactivate
```

What to do at the lab bench – final checklist

1. Read the practical aim and identify all modules from the kit.

2. Power OFF the Raspberry Pi before wiring.

3. Connect power, ground and signal wires according to the module labels and verified pinout.

4. Ask the instructor to verify the wiring before switching ON, especially for relay, fingerprint and RFID modules.

5. Boot the Pi and open Terminal.

6. Enable the required interface (I2C/SPI/UART/camera) if not already enabled.

7. Open the correct practical folder and save the exact code from the original PDF using the filename in this manual.

8. Install only the required dependencies, then run the program using the command provided.

9. Compare the output with the expected result and take a screenshot/photo if your lab record requires it.

10. Stop the program with Ctrl+C when necessary, switch off the Pi before removing wires, and return components safely.

Very important: Pin connections in this manual are standard examples for common modules, not a guarantee for every kit. The exact Raspberry Pi model, module revision, voltage regulator and connector labels must be checked physically. For Practical 4 and the rfid_pi5 dependency, your uploaded PDF does not provide enough information to give an exact reproduction.